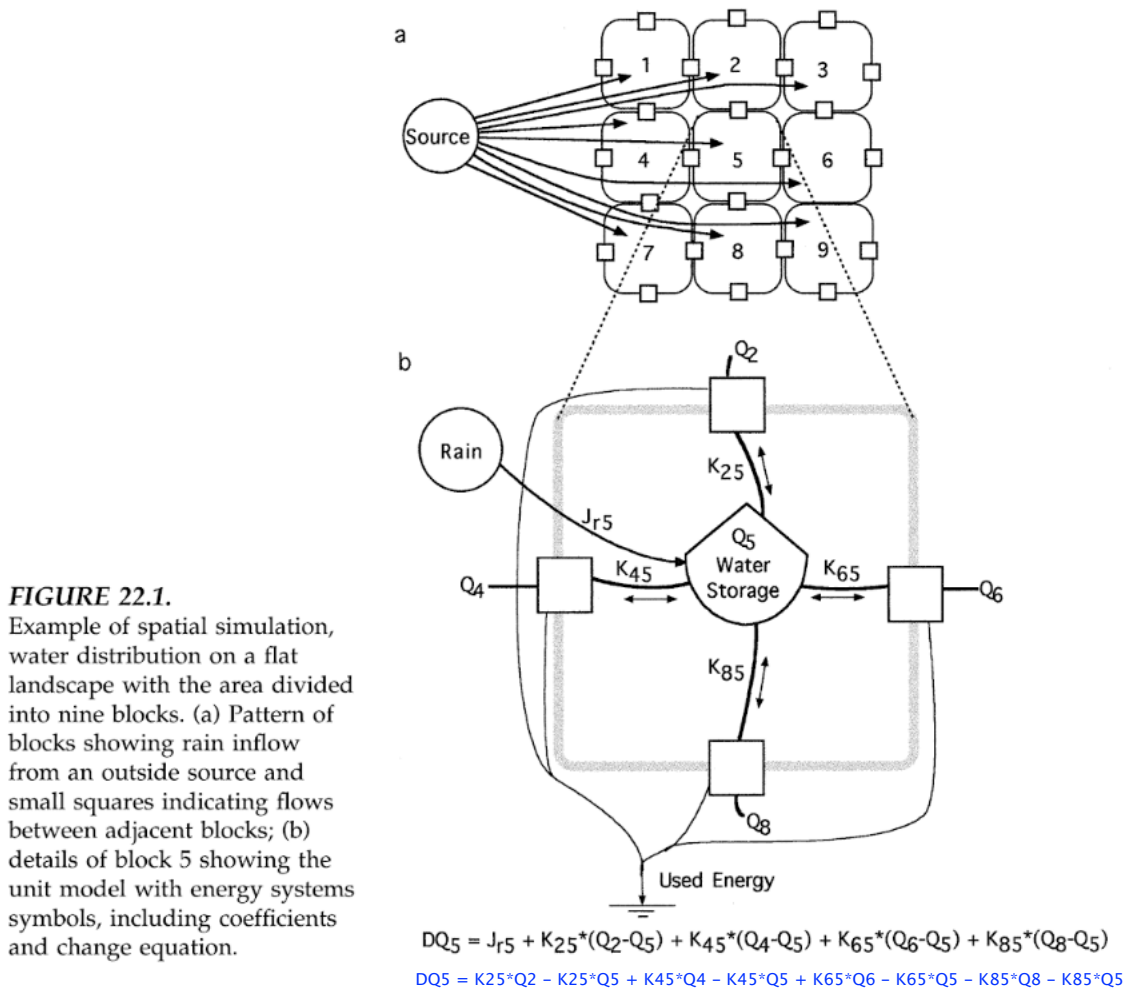




SIMPLE EXAMPLE: LINEAR WATER FLOWS

A very simple example is the model in Figure 22.1b for the flow of water in a flat, sandy landscape. In this case each block has the same minimodel. The repeating unit model in this case is a simple water storage, which exchanges water with the next block according to the difference in water quantity (Figure 22.1b). Rain enters each block and flows toward those blocks with less water, dispersing some energy from the frictional processes. The model is appropriate where flow is through a porous medium (such as sand) and thus appropriate for D'Arcy's law where flow is proportional to difference in pressure (a linear pathway), and pressure is proportional to the water stored Q .

The equation for each block is the same, although the coefficients (K) may be different (different friction). For the model in Figure 21.1 the program for simulation has nine equations, one for each of the nine storages.

A unit model with more pathways is required for simulating water flow on most landscapes, one including rain, percolation into deeper levels of the earth, evaporation, transpiration by the vegetation, runoff and other interblock flows, and quadratic flow where there is turbulent flow in channels. In the field of hydrology, commercial software packages help simulate water distribution and flow.

SPATIAL SELF-ORGANIZATION BY PULSING UNITS

With a more complex unit model (PULSE from Figure 15.4), Richardson (1987) studied the spatial patterns of energy production and consumption generated by spatial simulation. The unit model (Figure 22.2b) was simulated in 100 blocks (Figure 22.2a) with different kinds of interblock pathways, starting with the simple linear flow used in Figure 22.1. A digital-analog hybrid computer was used for the computations. Shading was used to show the magnitude of simulated quantities, with darker shading for larger quantities. Shown as a time sequence from left to right in Figures 22.2c and 22.2d are two sequences of shaded maps, one for the accumulated storage of producer products and the other for the storage of consumer assets during their surge of consumption and growth. This power-maximizing design automatically develops an organized center in spatial simulation. The consumer frenzy develops a hierarchical center because more flows converge to the blocks in the center than to those on the perimeter. Figure 22.2e has the graphs for one of the middle blocks. The peaks of the curves correspond to the dark shadings in Figures 22.2c and 22.2d. In the field of embryological development, Meinhardt (1982) simulated the early development of structures and patterns of organisms with spatial models of chemical concentrations.

SPATIAL SIMULATION SOFTWARE

In application to real systems, there may be many blocks and more complex unit models, so that the amount of computer

have subprograms for doing a general sensitivity test in one operation.

USING MATRIX EQUATIONS FOR SIMULATION

Where there are many units, all with similar equations, and where the equations are simple, the simulation programs can compute the iterations efficiently using matrix operations. To get the idea, consider the spatial simulation with many blocks in Figure 21.1. For one block on each iteration a new storage value Q_{t+1} is calculated as the old Q_t plus the inflow J plus the change due to flows between the blocks DQ . The change due to inter-block flows is the product of the K coefficients and the appropriate Q storages (= states) in the adjacent blocks. For block 5 in Figure 21.1a:

$$DQ_5 = K_{25} * Q_2 - K_{25} * Q_5 + K_{45} * Q_4 - K_{45} * Q_5 + K_{65} * Q_6 - K_{65} * Q_5 - K_{85} * Q_8 - K_{85} * Q_5$$

$$Q_{5t+1} = Q_{5t} + J_{r5} + DQ_5, \quad \text{where}$$

$$DQ_5 = K_{25} * Q_2 - K_{25} * Q_5 + K_{45} * Q_4 - K_{45} * Q_5 + K_{65} * Q_6 - K_{65} * Q_5 - K_{85} * Q_8 - K_{85} * Q_5$$

The equations for all of the blocks together can be represented with matrix mathematics so that all the blocks are calculated simultaneously on each iteration. The matrix expression for the system simulation is analogous in form to the equation for a single block and is shown in Figure 22.5 with block 5 highlighted and its coefficients shown.

FIGURE 22.5.
Matrix expressions for an iteration with the water model in Figure 22.1.

$$\begin{array}{c} \text{New State} \\ \text{Vector} \\ \underline{Q}_{t+1} \end{array} = \begin{array}{c} \text{State} \\ \text{Vector} \\ \underline{Q}_t \end{array} + \begin{array}{c} \text{Source} \\ \text{Vector} \\ \underline{J} \end{array} + \begin{array}{c} \text{Changes Due to} \\ \text{Flows between Blocks} \\ \text{Coef.} \\ \text{Matrix} \\ \underline{K} \end{array} * \begin{array}{c} \text{State} \\ \text{Vector} \\ \underline{Q}_t \end{array}$$

Multiplying the state vector Q_t by the coefficient matrix K generates a vector of changes DQ due to the flows between blocks. (In this operation the column vector is changed to a row vector and multiplied by each row of the matrix, starting at the top. Then the products in the row are added across to get the new Q values.) On each iteration, the program calculates new state values Q_{t+1} by adding the vector of source inputs J and the changes due to exchanges with surrounding blocks $K*Q$ to the state vector Q_t for time t . The state values Q_t for each iteration can be graphed or assigned by spatial graphics to show the distribution on a map.

Here the use of matrix equations in simulation was explained using a spatial example. However, matrix operations are used for other kinds of simulation wherever there are duplicate, repetitive iterations. For example, the Leslie matrix is a table of population numbers with age classes on one axis and the processes affecting each age class on the other axis. It is readily simulated by multiplying on each iteration the vector of age class numbers by the matrix of age-specific processes to obtain a new vector of age class numbers for each new time step. Grant (1986) explains with wildlife examples.

Programming languages (including BASIC) contain fundamental mathematics operations such as matrix addition, subtraction, multiplication, and inversion (analogous to division). Matrix operations are useful in spatial hydrology and fluid dynamics simulations where the same set of equations is used for large numbers of blocks.

On the right side of Figure 22.5, a state vector (list of Q storage values) was multiplied by a transition matrix (table of K coefficients) to obtain a new state vector. In many cases matrix algebra can calculate a substitute matrix that has the same result when multiplied by the vector, but has numbers only on the diagonals. These are called eigenvalues. Called an eigenvector, a set of Q values can be defined that will generate a multiple of itself when multiplied by the substitute matrix with its eigenvalues. This set of values in the eigenvector is the steady state. For example, the eigenvalues and eigenvector for the population numbers in a Leslie matrix are the stable age distribution (the distribution of numbers by age classes that generates itself in steady state).

For systems where the state change coefficients are linear (multiplied by only one state variable), matrix operations are

used to find steady states and criteria for stability. However, except when available energy is very small, real systems develop autocatalytic designs (maximum power principle) and are not linear. When equations are solved for the steady state (changes equal zero), many nonlinear terms become linear, but then the simplified equations only apply to small changes near equilibrium. Since all storages have depreciation (second law), there are negative terms on the diagonal of system matrices, a mathematical criterion of stability. Logofet (1993) provides matrix views of many ecological examples.

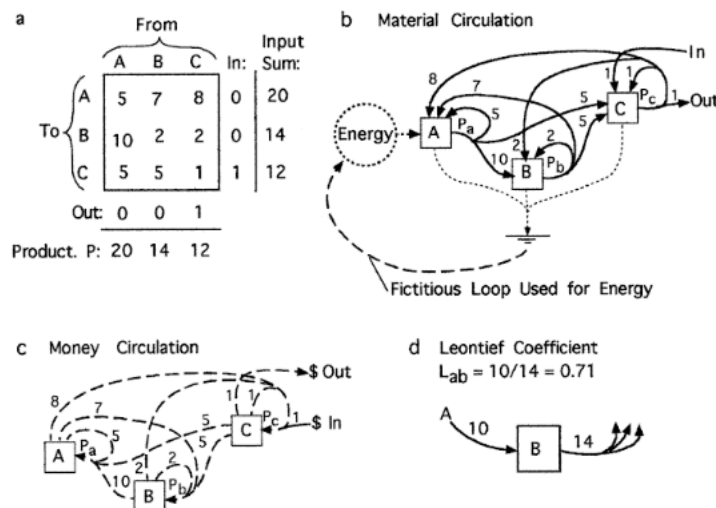
The rules for use of pathways and numbers on some other kinds of network diagrams are quite different from the energy systems language. For example, in one usage the arrowhead is directed from consumer to food, opposite to the energy flow. In energy systems language the arrowhead is always in the direction of the flows of energy, matter, and information—from food to consumer. The action of the consumer to get the food is shown as an autocatalytic feedback interaction (Figures 2.7 and 2.13a).

INPUT-OUTPUT SIMULATION

Simulations of systems can be done by matrix operations that extrapolate input-output relationships. The flows of materials or money between the units of a network can be represented with an *input-output table* (sometimes called a *transaction matrix*) (Figure 22.6a). Flows of materials are from the items listed at

FIGURE 22.6.

Example of an input-output table and its representation as an energy systems diagram. (a) Table of flows between three units; (b) network diagram with energy pathways added; (c) network diagram with reversed directions of flow, which are appropriate if flows are money; (d) calculation of a Leontief coefficient describing the flow relationship of two units.



the top to the items listed along the side. These flows can also be represented with an energy systems diagram that can show the energy and hierarchical relationships (Figure 22.6b). The diagonals of the matrix are flows circulating within the unit. The total of numbers added across is the total of inflow, whereas the total of columns adding down is the productive output. At steady state the sums of rows and columns are the same.

Energy circulation has been used in input–output tables by connecting the pathways of degraded energy (heat sinks) back as inputs of available energy (Figure 22.6b, which is an arrangement for mathematical purpose contrary to the second thermodynamic law).

Money paid for commodities flows in the opposite direction from the materials (Figure 22.6c). If the flows of money were numerically the same, the table in Figure 22.6a would suffice by swapping the words “from” and “to.” In an economic system units in the network are called *sectors* (industrial, agricultural, etc.). The human consumers are the *final demand sector* and are appropriately placed on the right, representing a superior hierarchical position (C in Figure 22.6c).

A Leontief coefficient (input per unit output) can be calculated for each input–output box as the ratio of the input flow to the total output (the production). For example, in Figure 22.6d the input flow from A to B is J_{ab} and the productive output of B is P_b . This Leontief coefficient is $L_{ab} = J_{ab}/P_b = 10/14 = 0.71$. These coefficients measure the structural linking of one unit to another by the flow. If it is believed that the system as a whole is going to grow 10%, without much change in the relationships of the various units, then matrix inversion methods can be used with the coefficient matrix to backcalculate what each intersector flow will become consistent with the increased total growth. Such calculations have been made for national economies with more than 600 boxes (sectors).

22.5 DISCRETE PROCESSES AND MODELS

Many processes and models have numerical steps. For example, stairs in a house have steps. The height of the levels where you place your foot are in a discrete series. With continuous

functions the values for a state can have any value. With discrete functions the values are in steps. Most of the models simulated in this book involved continuous functions, although some involved on-off switching actions (example: the FIRE program of Table 15.3).

LOGIC NETWORKS

Logic networks are arrangements of on-off units. For example an AND gate unit is on (has an output of 1) when its inputs are both on (have values of 1). An OR gate is on if either of its inputs is on. A NOT unit is on when the input is off, but off when the input is on. A threshold switch is on when the input of a continuous function is greater than a threshold value (example: water overflowing a river bank). Flip-flops are set by an input pulse of 1 and stay on until turned off by a reset pulse. Some systems have networks of these logic units. They are discrete systems since the values have only two alternative steps, on and off. Models of discrete processes can be simulated by writing a program with a sequence of logic actions. Simulation programs, including BASIC, have the common logic actions AND, OR, NOT, IF-THEN, etc., as part of the language.

Systems of continuous functions can be simulated with discrete networks with results that exaggerate responses. For example, a flip-flop unit can simulate the competition between two species that exclude each other in competition. Depending on input thresholds, there is one species or the other, but not both. Robertson and Muetzelfeldt (1991) use logic programming in ecological systems.

Hardware simulators are available in which you can connect various logic units with jumper wires so that the complex consequence of many units is simulated. The digital computer is such a discrete system built of enormous numbers of electrical logic circuits with values either on (logic 1) or off (logic 0). By having a huge number of such units in the extremely miniaturized network on the computer chip, the digital computer simulates continuous functions, carrying numbers to 40 significant figures.

CUMULATIVE ERROR IN DIGITAL SIMULATION

There is a tiny error that can accumulate in digital simulation of continuous functions, especially in growth processes. Where

TABLE A.1 (Continued)

```

{
Con1outarray[code] = -1;
Con1outarray[Force] = 0.0;
Con1outarray[Flow] = 0.0;
Con1outarray[transformity] = 0.0;
}
Else
{
Con1outarray[Flow] = Received1outarray[Flow];
}
If (Not Getpassedarray(Con2out, Received2outarray))
{
Con2outarray[code] = -1;
Con2outarray[Force] = 0.0;
Con2outarray[Flow] = 0.0;
Con2outarray[transformity] = 0.0;
}
Else
{
Con2outarray[Flow] = Received2outarray[Flow];
}

Inflow1 = Con1inarray[Flow];
Inflow2 = Con2inarray[Flow];
Inflow3 = Con3inarray[Flow];
Outflow1 = Q* Con1outarray[Flow];
Outflow2 = Q* Con2outarray[Flow];

DQ = inflow1 + Inflow2+inflow3 -Kd*Q - outflow1 -outflow2;
Q = Q + DQ* DeltaTime;
If (Q==0)
Q=0.000000001;
If (Q<0)
Q=0.000000001;

St1 = con1inarray[transformity];
St2 = con2inarray[transformity];
St3 = Con3inarray[transformity];

If (DQ/Q>0.05)
DEM = St1*inflow1 +ST2*inflow2 +st3*inflow3- ST*outflow1 +

If (Dq/Q<0.05 and Dq>0.0)
DEM=0;

If (Dq ==0.0)
dEM= 0.0;

If (DQ<0.0)
DEM=ST*DQ;
EM = EM + DEM*Deltatime;
If (EM ==0.0)
EM = 1;

If (EM <0.0)
EM = 1;

ST = EM/Q;
Con1inarray[force] = Q;
Con2inarray[force] = Q;
Con3inarray[force] = Q;

Con1outarray[transformity] = ST;

```

(continues)

TABLE A.1 (Continued)

```

Con2outarray[transformity] = ST;
Con1outarray[code] = 1;**code 1 is force out
Con2outarray[code] = 1;**code 1 is force out
Con1outarray[Force] = Q;
Con2outarray[Force] = Q;

Con1in = Passarray(Con1inarray);
Con2in = Passarray(Con2inarray);
Con3in = Passarray(Con3inarray);

Con1out = passarray(Con1outarray);
Con2out = passarray(Con2outarray);
}

** Initialize any simulation variables.
on initsim
{
Q = Initstore;
St = Inittransformity;
Kd = Calibdrain/Calibstore;
Em = Q*St;

Makearray(con1inarray,4);
Con1inarray[code] = -1;
Con1inarray[Force] = 0.0;
Con1inarray[Flow] = 0.0;
Con1inarray[transformity] = 0.0;

Makearray(con2inarray,4);
Con2inarray[code] = -1;
Con2inarray[Force] = 0.0;
Con2inarray[Flow] = 0.0;
Con2inarray[transformity] = 0.0;

Makearray(con3inarray,4);
Con3inarray[code] = -1;
Con3inarray[Force] = 0.0;
Con3inarray[Flow] = 0.0;
Con3inarray[transformity] = 0.0;

Makearray(con1outarray,4);
Con1outarray[code] = -1;
Con1outarray[Force] = 0.0;
Con1outarray[Flow] = 0.0;
Con1outarray[transformity] = 0.0;

Makearray(con2outarray,4);
Con2outarray[code] = -1;
Con2outarray[Force] = 0.0;
Con2outarray[Flow] = 0.0;
Con2outarray[transformity] = 0.0;
}

** User clicked the dialog HELP button.
on help
{
showHelp();
}

```

```
Real Con1inarray[], Con1outarray[];
Real Received1inarray[], Received1outarray[];
```

On another line type any variable or parameter names used within the block such as:

```
Real, Q, Water, K1, K2, DQ, DE, Inflow1, outflow1;
```

13. *Type program lines between the brackets.* Brackets are already provided for the several categories of message handling. Program statements must end with a semicolon. Capitals or lowercase letters are interchangeable. If you want to leave a remark in the program that tells you later what you were doing or why, type a `**` or `//` followed by the remark. The program ignores anything on the same line that follows `**` or `//`. You will need to program three message handling groups which can be in the script in any order:

On Createblock, which gives values to dialog boxes when they open.

On Initsim, which calculates the initial conditions before the run.

On Simulate, which has the program steps during the run.

14. *Scroll the script program space until On CreateBlock appears.* (Use the mouse pointer on the arrow along the right margin.) Type here whatever numbers you want the dialog boxes to have when they are first called to the screen. Type a parameter name (that will be used by the program) showing it equal to the number you want to appear. For example, type

```
{
WATERSTORE = 1000;
}
```

Whenever someone pulls the block from the library onto a worksheet screen, it will have the value 1000. Thereafter the user can change it if needed. If you put in nothing, the dialog box will be empty when it is clicked on.

15. Next scroll the script space until *On Initsim* appears. Here you should set initial conditions and create the input and output arrays that will be used:

```
{
  Makearray(con1inarray,4)
  Con1inarray[code] = -1;
  Con1inarray[Force] = 0.0;
  Con1inarray[Flow] = 0.0;
  Con1inarray[Transformity] = 0.0;

  Makearray(con1outarray,4)
  Con1outarray[code] = -1;
  Con1outarray[Force] = 0.0;
  Con1outarray[Flow] = 0.0;
  Con1outarray[transformity] = 0,0;

  Eq = 1000; **initial solar energy storage
}
```

16. Scroll the script space to find “*On Simulate.*” Type the simulation program between the brackets. Start with an IF statement as to what values the arrays should receive from connecting blocks and which should be set from the block being programmed. For example:

```
If (not getpassedarray(con1in, Received1inarray))
{
  Con1inarray[code] = -1;
  Con1inarray[force] = 0.0;
  con1inarray[flow] = 0.0;
  Con1inarray[transformity] = 0.0;
}
Else
{
  (Con1inarray[flow] = Received1inarray[flow];
  (con1inarray[transformity] = Recieved1inarray[transformity];
}
```

This statement sets input array positions to zero if nothing is received, but if received, the flow is given the value received from the upstream input.

A similar statement is used to receive downstream values shared by the output array:

```

If (not getpassedarray(con1out, Received1outarray))
{
  Con1outarray[code] = -1;
  Con1outarray[force] = 0.0;
  con1outarray[flow] = 0.0;
  Con1outarray[transformity] = 0.0;
}
Else
  Con1inarray[flow] = Received1inarray[flow];

```

17. *To write short equations for action, transfer information received to shorter internal names or letters.* For example:

```

Jin = Con1inarray[flow];
Stin = Con1inarray[transformity];
Jout = Con1outarray[flow];

```

18. *Write the equations using the short labels.* For example, if the change in storage is the sum of inflows minus outflows:

$$DQ = Jin - Jout - K1 * Q;$$

and the new value of storage is:

$$Q = Q + DQ * \text{deltatime};$$

Deltatime is the iteration interval DT.

For emergy storage (Eq), storage Q, and solar transformity received in (Stin), the transformity of storage St is:

$$St = Eq / Q;$$

and the change in emergy storage sums the products of flows and transformities:

$$DEq = Stin * Jin - St * Jout;$$

and the new value of stored emergy is:

$$Eq = Eq + DEq * \text{deltatime};$$

B.6 USE OF SINE WAVES FOR VARYING INPUTS

Some inflows to a system that vary regularly can be programmed using sine waves. This is especially useful for the daily and seasonal variations in sunlight that follow sine waves on clear days (Figure B.1). Seasonal rainfall, evaporation, temperature, etc., can also be programmed this way. Giving a regular up and down input is a way of studying a system's response and comparing with others.

As a wheel with a radius of one rotates, a nail generates a sine wave. The height of a nail on the rim goes from zero to +1 to zero to -1 and back to zero, where measurements are made from the level of the axle (see Figure B.1a). A right triangle is formed between the nail, the axle, and the horizontal line of the axle.

The sine of an angle is the opposite side over the hypotenuse of a right triangle. As the wheel makes one full revolution, the sine of the angle A (the angle between the nail spoke and the horizontal) goes from 0 to +1 to 0 to -1 to 0 again. When the

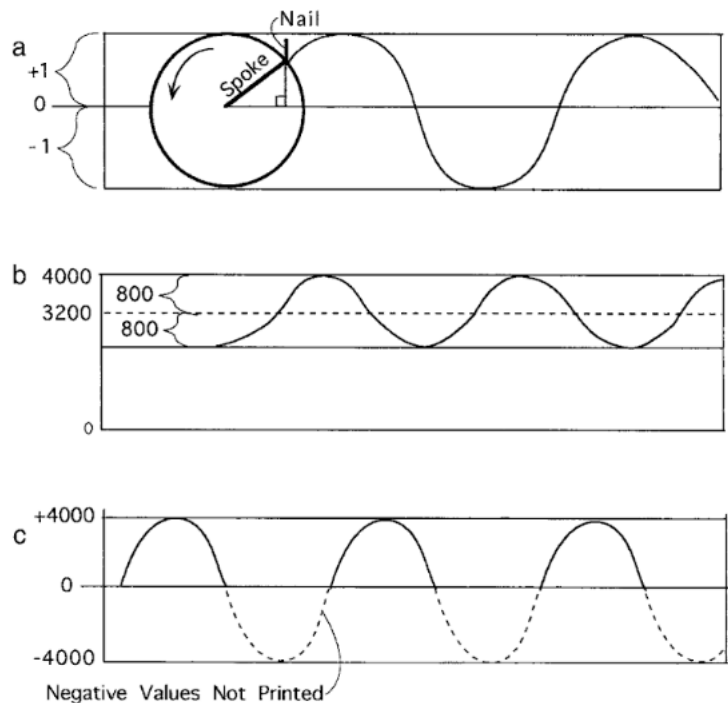


FIGURE B.1.

Sine waves for supplying diurnal and seasonal solar inputs.

(a) Sine wave viewed as a line generated by a nail on a rotating wheel, with amplitude plus and minus 1, as generated by BASIC; (b) graph for representing annual inputs using an equation: $Y = 3200 + 800 \sin(T \cdot 0.017)$; (c) graph representing diurnal inputs using the equation: $Y = 4000 \sin(T \cdot 0.26)$.

value of the sine A is graphed with time (or the path of the nail up and down graphed with time), a sine wave results.

The angle between the nail spoke and the horizontal as the wheel turns a whole revolution goes from 0 to 90 to 180 to 270 to 360 degrees (0 degrees again). One can also represent the angle in radians. A radian is the arc of the circle the length of the radius. When the wheel makes a full revolution, it passes over a circumference of 2π radii. A full revolution of 360 degrees is an angle of 2π radians. Since π is 3.14, 2π is 6.28. A full circle that is a full sine wave cycle is 6.28 radians.

Sine waves may be used as a source-driving functions. The ups and downs of solar insolation during the year can be represented with a sine wave. The sunlight during the day can be represented with half a sine wave. The BASIC language has the sine function $Y = \text{SIN}(X)$, where X is the angle in radians. As X goes from 0 to 6.28, Y goes through a full cycle (0 to +1 to 0 to -1 to 0 again). To fit a sine wave to a program, substitute time (T) for X . Then multiply or divide T by whatever number is necessary to scale the sine wave to the time needed. For example, if a simulation is being run in days, using the sine function without scaling would generate one cycle every 6.28 days.

EQUATION FOR AN ANNUAL SINE WAVE

To make the sine wave cycle in 1 year (365 days), multiply the time by $(6.28/365 = .017)$. The equation becomes $Y = \text{SIN}(T*.017)$. This sine wave will go from +1 to -1 and back in 1 year. To scale this up or down, multiply by the range of sunlight (or whatever is being represented by the sine wave). If this range is R , where R is the amount that the quantity (of sunlight) varies from its average V , then the equation becomes $Y = R*\text{SIN}(T*.017)$.

To move the graph completely into the positive range, add an amplitude equal to the average value of sunlight or whatever is being represented. If this average value is V (like the average per day for a year), then the equation for a seasonal source results: $Y = V + R*\text{SIN}(T*.017)$. See the example shown in Figure B.1b.

EQUATION FOR A DAILY SINE WAVE SOURCE

If the time unit is in hours and there are 24 hours a day, then scale the sine function so that one cycle takes 24 time units.